

```
#!/usr/bin/env python3
"""
Tanglish++ Language Interpreter
Official Version 1.0.0
Creator: Vignesh Arumugam
A Tamil-centric programming language with
Python-compatible syntax
"""

import sys
import re

KEYWORDS = {
    'unmai': 'TRUE_KW',
    'poi': 'FALSE_KW',
    'ethumeilla': 'NONE_KW',
    'oruvelaerunth': 'IF_KW',
    'oruvelaillina': 'ELIF_KW',
    'athuvuillina': 'ELSE_KW',
    'neruthu': 'BREAK_KW',
    'neecontinue': 'CONTINUE_KW',
    'apro': 'AND_OP',
    'illana': 'OR_OP',
    'illa': 'NOT_OP',
    'fun': 'DEF_KW',
    'thirupikudu': 'RETURN_KW',
    'lineda': 'LAMBDA_KW',
    'ethupannatha': 'PASS_KW',
    'eduthuko': 'IMPORT_KW',
    'engaerunthu': 'FROM_KW',
```

```

'sollu': 'PRINT_KW'
}

TOKEN_SPECIFICATION = [
    ('NUMBER', r'\d+'),
    ('ASSIGN', r'='),
    ('NAME', r'[a-zA-Z_]\w*'),
    ('NEWLINE', r'\n'),
    ('SKIP', r'[ ]+'),
    ('MISMATCH', r'.'),
]

def tokenize(code):
    """Convert raw Tanglish++ source code
    into token stream"""
    tokens = []
    for line in code.split('\n'):
        line = line.strip()
        if not line or
line.startswith('#'):
            continue
        line_tokens = []
        tok_regex = '|'.join(f'(?P<{name}>
{pattern})' for name, pattern in
TOKEN_SPECIFICATION)
        for mo in re.finditer(tok_regex,
line):
            kind = mo.lastgroup
            value = mo.group(kind)
            if kind == 'NUMBER':
                value = int(value)
            elif kind == 'NAME' and value
in KEYWORDS:
                kind = KEYWORDS[value]

```

```

        elif kind == 'SKIP':
            continue
        elif kind == 'MISMATCH':
            raise
SyntaxError(f'Unexpected character
{value!r}')
        line_tokens.append((kind,
value))
        if line_tokens:
            tokens.append(line_tokens)
        return tokens

class TanglishPlusPlus:
    """Main interpreter engine for
Tanglish++ programs"""

    def __init__(self):
        self.variables = {}

    def eval_value(self, token):
        """Evaluate a single token to its
Python equivalent"""
        kind, val = token
        if kind == 'TRUE_KW':
            return True
        if kind == 'FALSE_KW':
            return False
        if kind == 'NONE_KW':
            return None
        if kind == 'NUMBER':
            return val
        if kind == 'NAME':
            return self.variables.get(val,
f"NameError: '{val}' is not defined")

```

```

        return val

    def run(self, code):
        """Execute Tanglish++ code"""
        lines = tokenize(code)
        idx = 0
        skip_else = False

        while idx < len(lines):
            line = lines[idx]
            first_tok_type = line[0][0]

            # Variable assignment: name =
            value
            if first_tok_type == 'NAME' and
            len(line) > 1 and line[1][0] == 'ASSIGN':
                var_name = line[0][1]
                self.variables[var_name] =
                self.eval_value(line[2])

            # Print statement: sollu value
            elif first_tok_type ==
            'PRINT_KW':

            print(self.eval_value(line[1]))

            # If statement: oruvelaerunth
            condition
            elif first_tok_type == 'IF_KW':
                condition_val =
                self.eval_value(line[1])
                if condition_val is True:
                    idx += 1
                    skip_else = True

```

```

        continue
    else:
        idx += 1
        skip_else = False

    # Else statement: athuvuillina
    elif first_tok_type ==
'ELSE_KW':
        if skip_else:
            idx += 1
        else:
            idx += 1
            continue

    # Pass statement: ethupannatha
    elif first_tok_type ==
'PASS_KW':
        pass

    idx += 1

def main():
    """Entry point for the interpreter"""
    if len(sys.argv) < 2:
        print("Usage: python
tanglish_plus_plus.py <filename.t>")
        print("\nTanglish++ v1.0.0 - Tamil
Programming Language")
        sys.exit(1)

    filename = sys.argv[1]

    if not filename.endswith('.t'):
        print("Error: Tanglish++ source

```

```
files must end with '.t'")
    sys.exit(1)

    try:
        with open(filename, 'r',
encoding='utf-8') as file:
            code = file.read()
            interpreter = TanglishPlusPlus()
            interpreter.run(code)
    except FileNotFoundError:
        print(f"Error: File '{filename}'
not found.")
        sys.exit(1)
    except SyntaxError as e:
        print(f"SyntaxError: {e}")
        sys.exit(1)

if __name__ == "__main__":
    main()
```